Measurement discipline and relevant information:

Measurement Discipline (System Configuration):

- System Specs:
- CPU: Intel Core i5 1135G7, 4c/8t, 400MHz-4.20GHz
- --> Turbo boost disabled via Intel P_State governor modifications
- --> Deliverables #5-8 pinned to thread 0 (working set sweep, cache miss impact, TLB miss impact, failed benchmark)
- --> Cache working sets were based on cache size information pulled from Intel Ark website (along with some linux commands), see further below
- RAM: 8GB LPDDR4X 4267MHz (Dual Channel)
- OS: Linux Debian Trixie 13.7.0
- Compiler/Tools:
- GCC 14.2.0 (Default with Debian 13)
- Governor: intel cpu_freq
- VSCode IDE

Datatype chosen: Long (int) [usually 64-bit]

Cache Working set sizes: (these sizes fit within these cache hierarchies)

- For long (usually 64-bit):
- --> 2,048 Longs = ~16KiB (L1-I, per core)
- --> 4,608 Longs = ~36KiB (L1-D, per core)
- --> 131,072 Longs = ~1MiB (L2, per core)
- --> 786,432 Longs = ~6MiB (L3, shared)
- --> 8,388,608 Longs = ~64MiB (DRAM, no caching)

According to Intel's website for the i5 1135g7 + some linux commands, I was able to derive the cache hierarchy for this CPU:

- L1-I: 128KiB (32KiB per core)
- L1-D: 192KiB (48KiB per core)
- L2: 5MiB (1.25MiB per core)
- L3 "Smart": 8MiB (shared)

Strides:

- For #1-4: 8 Longs (64B), 32 Longs (256B), 128 Longs (1024B)
- For #6: 0 Longs (no stride), 8 Longs (64B), 32 Longs (256B), 128 Longs (1024B)
- For #7: 1 Long (8B), 512 Longs (4096B), 262,144 Longs (2MiB)

Read/Write Ratios:

- 100% Reads

- 66.6% Reads / 33.3% Writes (as close as I could get to 70/30)
- 50% Reads / 50% Writes
- 100% Writes

Core compromise: Intel MLC:

- Intel MLC is limited in how it handles system configurations, so I cannot disable SMT/Hyperthreading when running it, nor can I pin cores/threads to it either.
- Intel MLC fails in certain custom argument configurations, hence will be labeled N/A if needed
- Intel MLC cannot track bandwidth when running granularity sweeps, it also cannot track bandwidth for stride sweeps of random access nature.
- Here is some core information when running Intel MLC:
- --> Turbo Boost was disabled
- --> Idle/Sleep C states were disabled
- --> Frequency was pinned to 1.40GHz
- --> Hyperthreading/SMT is still enabled
- --> No core/thread pinning was introduced

A secondary compromise: perf:

- perf library cannot measure L2 hits and misses without a custom MSR, which makes it even less reproducible as it varies not only by vendor (Intel,), but also by microarchitecture, so that means that even if you have an intel cpu, if it is of a different generation or architectural tier, it will potentially have a different L2 metric, so it would break across too many systems

Pre-experiment Hypothesis:

- I expect that the smaller faster caches will result in higher throughput and effective bandwidth when minimizing strides.
- --> When sweeping through working set sizes: It will transition into larger level caches, which means that the hitrate of L1 cache will go dramatically down, and once working set sizes surpass the size of a certain level of cache, that cache level's hitrate will nosedive.
- --> Impact of cache misses: More cache misses will dramatically decrease effective bandwidth and increase latency.
- --> Impact of TLB misses:
- --> Suspected benchmark failure: Core/thread pinning will not work on intel MLC

Data collected:

- For #1-4, I measured a couple things:
- --> Bandwidth (MB/s or GB/s)
- --> Idle Latency (ns)
- --> Loaded Latency (ns)
- For #5-7, I measured a few things:

- --> CPU Instructions (Count)
- --> CPU Cycles (Count)
- --> L1 Hitrate (%)
- --> LLC Hitrate (%) (usually L3 cache)
- --> L1 Misses (Count)
- --> LLC Misses (Count) (usually L3 cache)

AI Use Note:

- Tools used: Copilot (Desktop), Copilot (Bing), Gemini (Online)
- Copilot was used for:
- --> Initial code generation for benchmarking harness and perf calls (I modified it quite a bit)
- --> Looking up documentation regarding: performance profiling for intel_pstate governors, Intel MLC quirks, Intel MLC custom commands
- --> Support in the scripting (how to write to file output, how to elevate permissions, etc)
- --> Code verification (2nd opinion, as I check first)
- --> 2nd opinion on data results (I check first)
- Gemini was used for:
- --> Documentation on how to generate random numbers and random vectors
- --> Assisting with required libraries for perf
- Note: There are comments for parts of my code where I mention if it is AI generated, and what prompts were used. Although I still modify a lot of the AI-generated code

Another important source: https://linux.die.net

- I used the cpupower documentation there when configuring my cpu for the benchmarking script.

How to use:

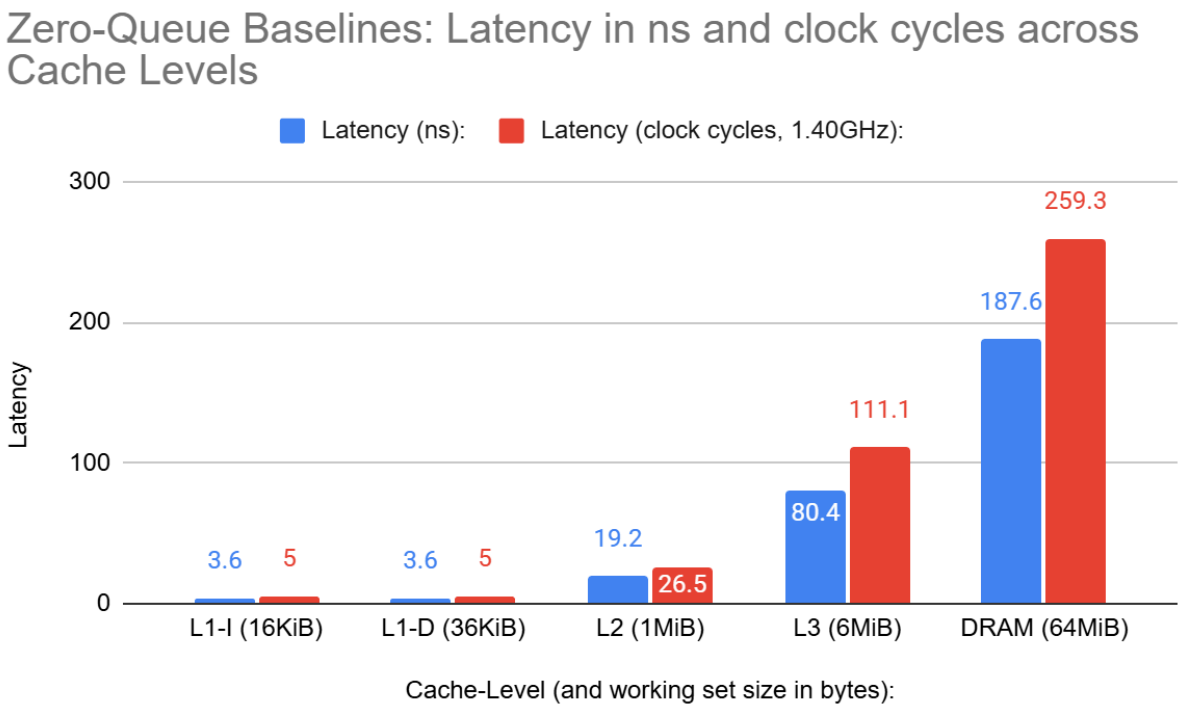
- Download the required files and make sure they are all in the same folder directory:
- Required files:
- --> project2cache.cpp
- --> benchmarkscript.sh (you need sudo priviledges for this)
- --> you should have intel mlc here too
- Optional files: (these are for support)
- --> HowToFixCoreAllocationScript.txt (if your OS is masking cores, use this, tested on Debian)
- Step 1: Run the benchmarkscript.sh file in terminal
- Step 2: Look at the result files it generates for you (.txt and .csv files)

Report + Required Experiments and Plots:

Pre-experiment hypothesis: I expect that the smaller faster caches will result in higher throughput and effective bandwidth when minimizing strides.

- --> When sweeping through working set sizes: It will transition into larger level caches, which means that the hitrate of L1 cache will go dramatically down, and once working set sizes surpass the size of a certain level of cache, that cache level's hitrate will nosedive.
- --> Impact of cache misses: More cache misses will dramatically decrease effective bandwidth and increase latency.

1. Zero-queue baselines:

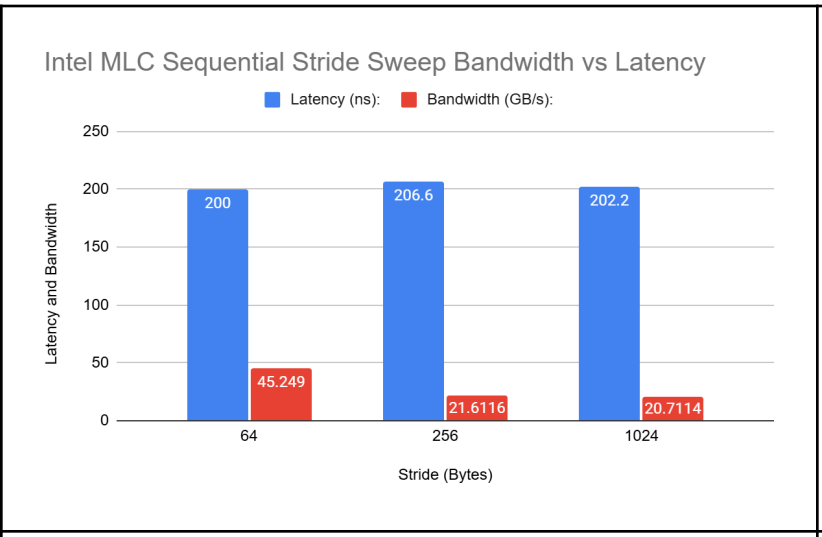


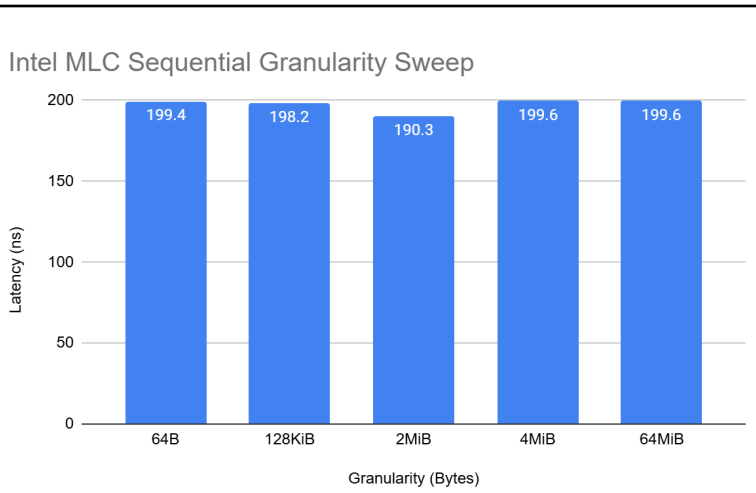
Cache-Level:	Working Set Size (B):	Latency (ns):	Latency (clock cycles, 1.40GHz):
L1-I (16KiB)	16384	3.6	5
L1-D (36KiB)	36864	3.6	5
L2 (1MiB)	1048576	19.2	26.5
L3 (6MiB)	6291456	80.4	111.1
DRAM (64MiB)	67108864	187.6	259.3

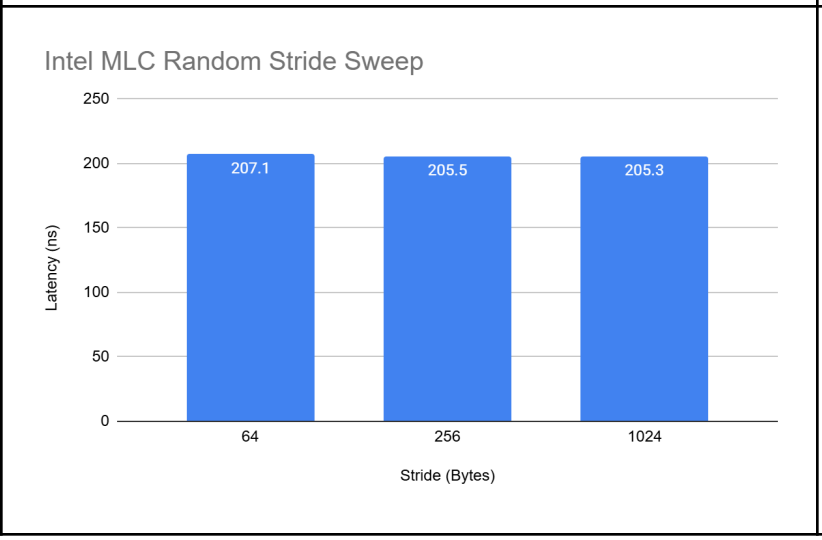
As the working set exceeded each cache level, the measured latency increased significantly. This behavior is expected because larger cache levels have greater capacity and more complex lookup structures, resulting in higher access latency. Once the working set exceeded the LLC

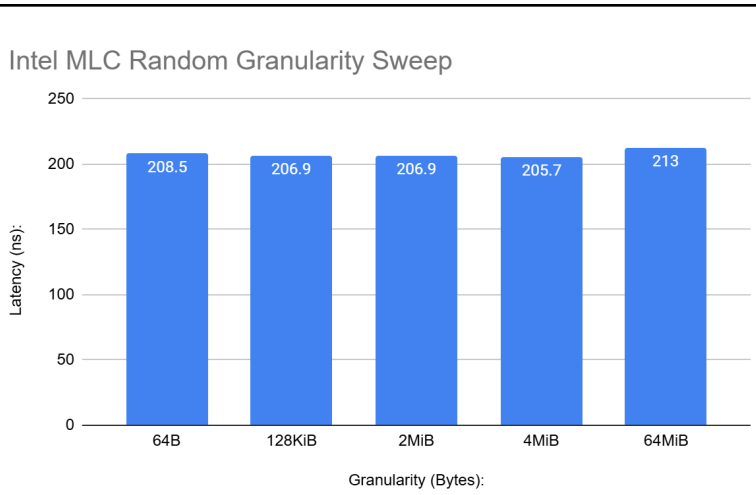
and accesses reached DRAM, latency increased substantially due to the much slower access time of main memory compared to on-chip caches.

2. Pattern and granularity sweep:







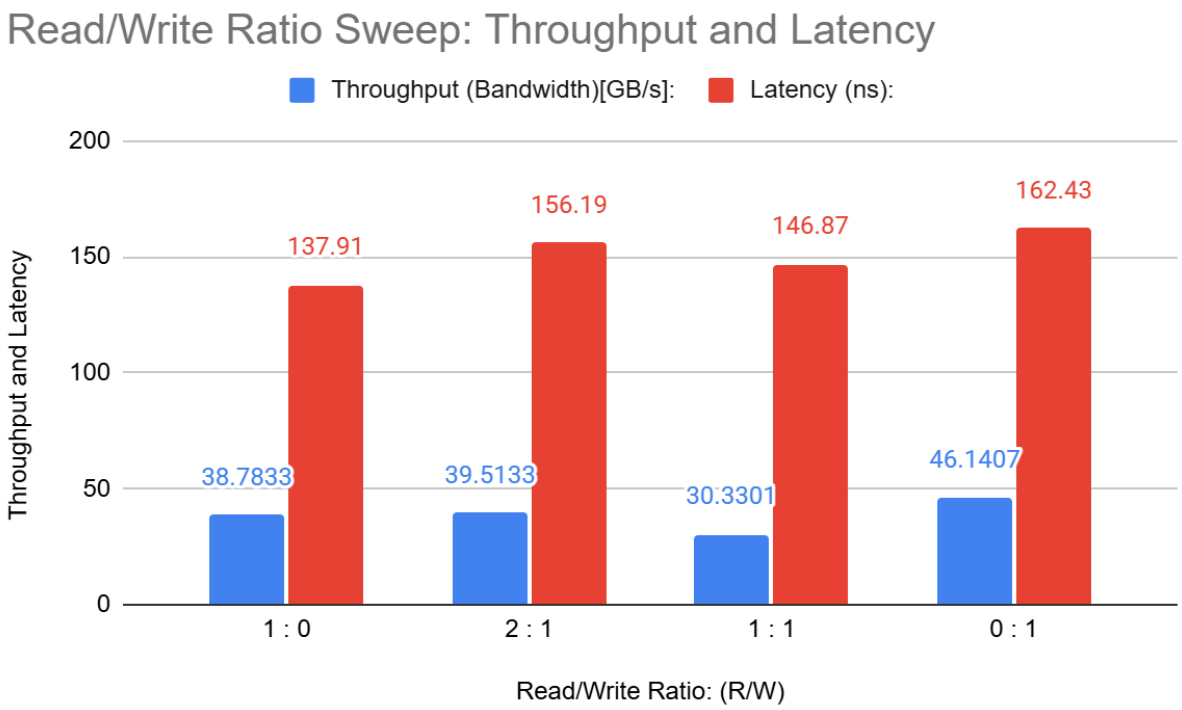


Intel MLC does not expose bandwidth measurements for sequential granularity sweeps or random-access granularity sweeps, so only latency measurements were collected for those experiments. Overall, varying access granularity and stride produced relatively small changes in measured latency. Random-access patterns consistently exhibited slightly higher latency than sequential-access patterns, which is expected because they reduce spatial locality and make hardware prefetching less effective.

For sequential stride sweeps, bandwidth decreased substantially when stride size increased from 64 bytes to 256 bytes (~-52.23%), but only decreased slightly between 256-byte and 1024-byte strides (~-4.16%). Larger strides reduce the amount of useful data obtained from each cache line fetch, reducing effective bandwidth even when latency remains relatively unchanged.

The relatively small latency differences suggest that performance was dominated by DRAM access time rather than cache-level effects. This is reasonable because a 128 MiB working set was used, placing the workload well beyond the capacity of the processor caches.

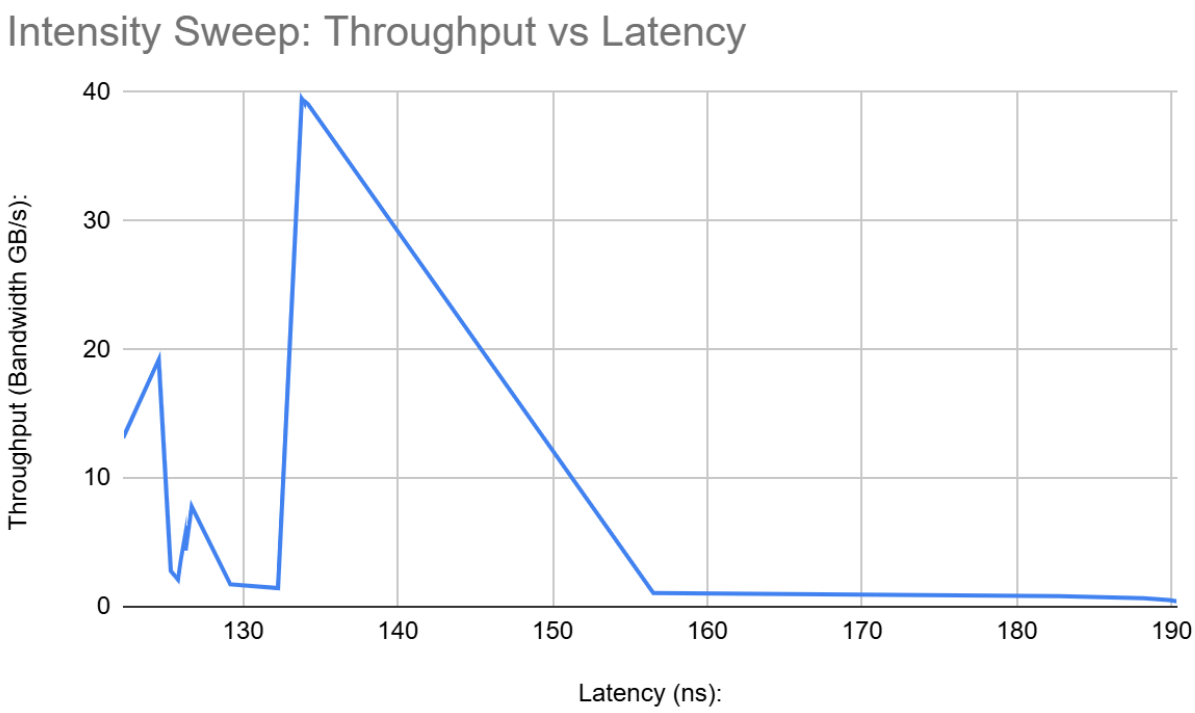
3. Read/write mix sweep:



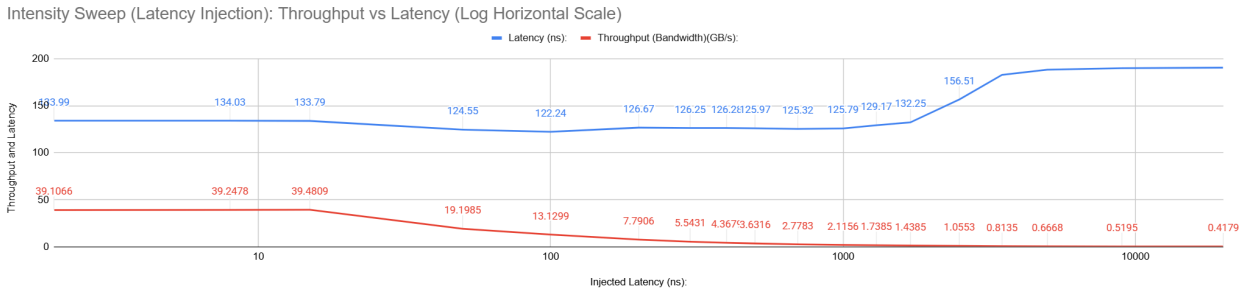
Both throughput and latency were highest for the 100% write workload. While throughput and latency varied across read/write ratios, the differences were relatively modest. The 100% read workload produced the lowest latency, while the 100% write workload achieved the highest throughput. The 50/50 read/write configuration yielded the lowest throughput overall.

The relatively small variation between read/write mixes suggests that the benchmark was primarily memory-bound rather than compute-bound. In other words, memory-system behavior had a greater effect on performance than arithmetic execution throughput.

4. Intensity sweep:



Throughput reached its maximum value at approximately 133.79 ns of latency. Beyond this point, additional queueing increased latency without producing meaningful improvements in throughput. This point represents the "knee" of the curve, where memory-system utilization is near its maximum and further increases in concurrency primarily increase waiting time rather than useful work. This relationship is more clearly illustrated in this other diagram:



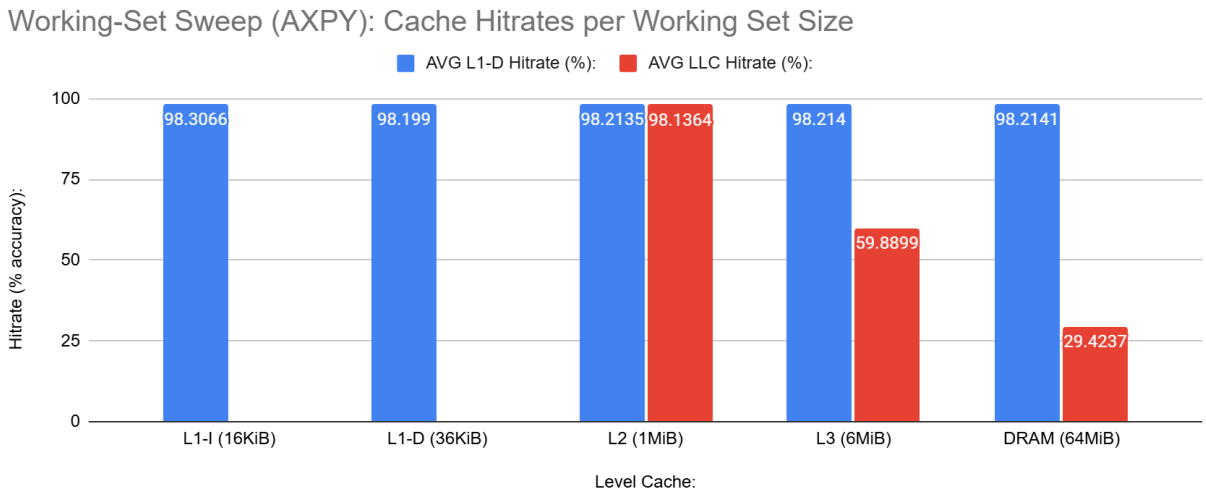
The earlier plot relates to little's law, as it contains the throughput and lead time.
Here is little's law and how my data ties into it:

- L = T x W
- Where:
    - W = Latency
    - T = Throughput (Bandwidth)
    - L = Average number of outstanding requests

    - → Concurrency or Queue-Length
    - Since bandwidth is data transferred per second, and latency is how long it takes for data to travel, then the resulting L is the amount of data that needs to be transferred at a time to achieve the throughput bandwidth.
- So at our knee, L = 39.4809GB/s * 133.79ns = 5.282 KB of requests at a time

As throughput increases, additional outstanding memory requests are required to sustain that throughput. Near the knee, the memory system approaches saturation, causing latency to increase rapidly while throughput gains become minimal.

5. Working-set sweep:



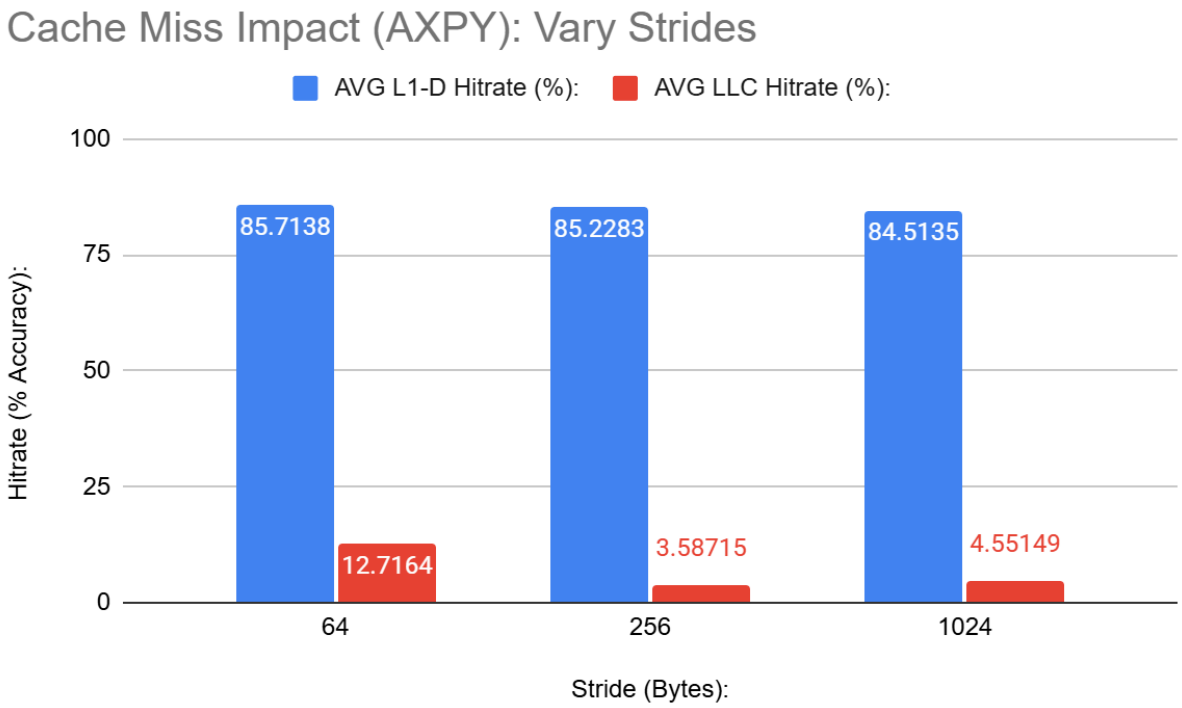
L1 cache hit rates remained consistently high across all working-set sizes. LLC hit rates did not appear until the working set reached L2-scale sizes because accesses were still being serviced by smaller cache levels. As the working set continued to grow, LLC hit rates declined because larger portions of the workload could no longer reside within the cache hierarchy.

Once the working set exceeded LLC capacity and entered the DRAM-resident region, LLC hit rates dropped substantially because much of the data needed to be fetched from main memory instead of being supplied by cache.

Initially, I expected L1 hit rates to decrease dramatically once the working set exceeded L1 capacity. However, the sequential nature of the AXPY kernel produces a streaming access pattern that is highly predictable. Modern hardware prefetchers can exploit this predictability to fetch future cache lines before they are needed, maintaining a very high L1 hit rate even when the overall working set is much larger than the L1 cache.

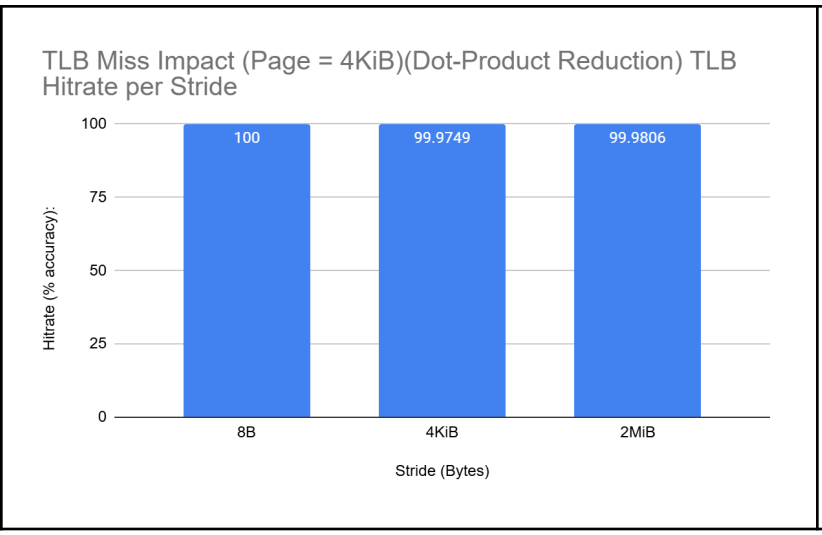
L2 hit rates were not included because L2 performance monitoring events are architecture-specific and are not exposed through the portable performance-counter interfaces used in this project. Excluding L2 metrics improves the reproducibility of the benchmark across platforms.

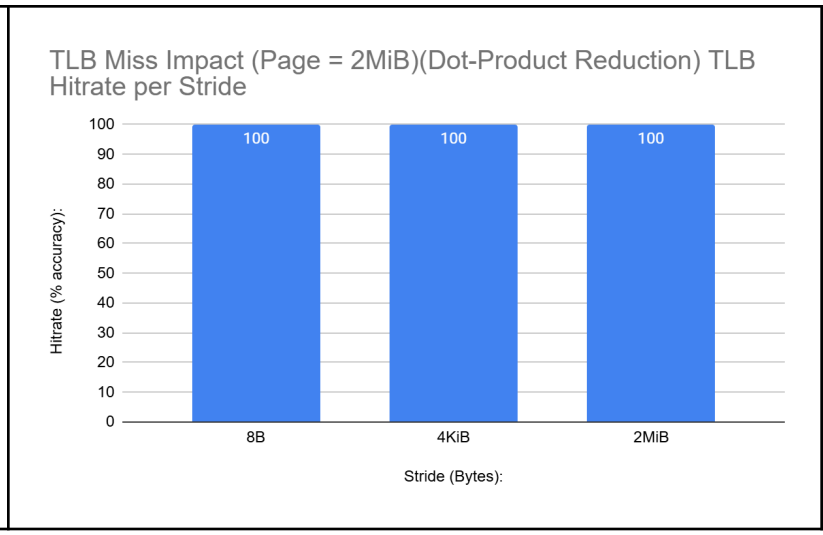
6. Cache-miss impact:



L1 cache hit rates remained relatively stable across all tested stride sizes. In contrast, LLC hit rates generally decreased as stride size increased, although a small increase was observed at the 1024-byte stride. One possible explanation for the LLC hitrate dropping is that larger strides decrease spatial locality, hence there is a lower chance of the next data being found in the LLC cache.

7. TLB-miss impact:





This result was surprising to me, as the hitrate didn't change (within margin of error). I would suspect that increasing the strides would reduce the hitrate as it would require more TLB table walks, but it didn't.

I suspect that the tested workload wasn't putting sufficient pressure on the TLB to overflow the TLB cache capacity. Modern processors can often have multi-level TLBs that can reduce the observed miss rates.

8. Benchmark pitfall:

The full table may be useful for demonstration:

Page Size (Bytes):	Stride (Bytes):	Stride (Bytes):	Avg Data TLB Hitrate (%):	Avg Data TLB Miss Count: (Optional)	Avg CPU Cycles: (Optional)	
4096	8	8B	100	2	74192509	(4KiB)
	4096	4KiB	99.9749	49	978223	
	2097152	2MiB	99.9806	0	3427	
2097152	8	8B	100	0	73804431	(2MiB)
	4096	4KiB	100	0	964708	
	2097152	2MiB	100	0	2442	

My TLB miss impact benchmarks had some pitfalls/failures. It recorded ~100% TLB hitrate across different stride sizes. I suspect that the TLB hitrate is for all levels of the TLB, which can be at or near 100%. A TLB benchmarking harness that extracts different levels of TLB hitrates may prove to be more useful. Or maybe utilizing larger strides or pages may stress the TLB more.

The TLB miss count is exceptionally low, and we can also see that the CPU cycles decreases dramatically as our strides increase. This could suggest that there are less elements to look from, so that the hit rate is less likely to increase.